V O L T S L O T

VoltSlot

EV Charging Station Management and Slot Reservation System

Project Documentation Report

Student name	[Enter student name]
USN	[Enter USN]
Department	[Enter department]
College	[Enter college name]
Project guide	[Enter guide name]
Academic year	[Enter academic year]

Prepared from the VoltSlot source code · 2026-10-14

Contents

Abstract

Introduction

Existing and Proposed Systems

Software and Hardware Requirements

System Architecture

System Modules

Database Design

API Documentation

Implementation Details

User Manual

Testing and Results

Security Considerations

Limitations and Future Enhancements

Conclusion

References

Abstract

VoltSlot is a web application for finding electric-vehicle charging stations, publishing scheduled charging slots, reserving available slots, and managing station operations.

The system uses a React single-page frontend and an Express/Node.js REST API backed by MongoDB through Mongoose. Customers can register, authenticate, browse and filter station listings, view schedules, place a reservation, review their own booking history, and cancel eligible bookings. Administrators have a separate portal for station and slot maintenance, booking oversight, and operational analytics. The backend validates inputs, verifies JWT bearer tokens, requires customer role for reservation creation, applies role checks to administrator and customer analytics operations, restricts booking reads/actions by ownership where applicable, and atomically claims a specific available slot against competing reservations.

This report documents the implementation visible in the repository and the automated tests executed while generating it. Personal/college fields remain editable placeholders. No live database mutation or application screenshot is represented as having occurred.

1. Introduction

Background

EV drivers need reliable information about charging locations and times, while station operators need a way to maintain location details and availability. VoltSlot models stations and scheduled charging slots and connects a reservation to a customer, station, and slot.

Problem statement

Without a shared directory and booking workflow, customers may not know where future capacity exists, while administrators need a controlled interface to maintain station data, publish slots, and oversee reservations. VoltSlot provides a single web application for these tasks.

Motivation

The project demonstrates a full-stack implementation of authenticated role-specific workflows, REST API validation, document modelling, and automated tests for common authorization and scheduling cases.

Objectives

- Provide customer registration, sign-in, profile editing, and a customer dashboard.
- Allow station discovery with text, city, connector, operational-status, availability, and optional proximity filters.
- Allow customers to reserve available future slots and manage eligible bookings.
- Give administrators station, slot, and booking management plus summary analytics.
- Enforce authentication, role policies, input constraints, booking ownership, and reservation conflict controls on the API.

Scope and limitations

Implemented scope includes the browser UI, REST API, MongoDB/Mongoose persistence, JWT authentication, role-gated portals, station/slot/booking flows, analytics, and automated unit/API/browser tests.

This repository does not implement payment processing, live charger telemetry, notification delivery, or a deployment workflow. Browser tests stub API responses and backend tests use mocked Mongoose operations; neither proves persistence against a production database.

2. Existing System and Proposed System

Existing-system context

For a project report, the baseline is the common fragmented process of consulting separate station listings and coordinating availability/reservations manually. This is contextual framing, not a measured comparison against a named installed competitor or prior VoltSlot release.

Limitations addressed

- Station descriptions, connectors, location, and schedule information may be managed in different places.
- Customers can lack a consistent view of upcoming availability and booking history.
- Administrators need separated tools for operational data and customer booking review.
- Manual reservation coordination can leave race conditions or unclear ownership unless the service validates and claims a slot.

Proposed solution

VoltSlot provides a customer station directory and booking workflow alongside an administrator portal. The backend exposes validated station, slot, booking, account, and dashboard endpoints; MongoDB stores the system records, and indexes support lookups and confirmed-slot uniqueness.

Advantages

- Role-specific navigation and workflows.
- Station search and schedule visibility in one interface.
- Server-side reservation checks and a database uniqueness constraint for confirmed bookings per slot.
- Ownership-aware customer booking retrieval and cancellation rules.
- Admin summaries, booking-status charts, and most-booked station analytics.

3. Software and Hardware Requirements

Category	Implementation / recommendation
Frontend	React 19, React Router 7, Vite 8, Axios, Bootstrap 5, Recharts and React Icons (package manifest dependencies).
Backend	Node.js (runtime exercised here: v24.19.0), Express 5, Mongoose 9, bcrypt, jsonwebtoken, express-validator, express-rate-limit, Helmet, CORS.
Database	MongoDB configured by backend environment variable MONGODB_URI; server startup connects to MongoDB before listening.
Development / tests	npm, Vite, Vitest, React Testing Library, Playwright, Node built-in test runner, Supertest.
Browser	Current desktop/mobile browser with JavaScript and localStorage enabled; internet/reachability to the configured API and database is required for live operations.

Category	Implementation / recommendation
Practical local hardware (recommendation)	64-bit computer, 4 CPU cores, 8 GB RAM, and at least 2 GB free disk for source/dependencies/build output. These are practical recommendations, not application-enforced minimums.
Network / security configuration	Reachable MongoDB deployment and environment configuration for MONGODB_URI and JWT_SECRET; configure CORS_ORIGINS when the frontend is served from a non-local origin. Do not place credentials in reports or source control.

4. System Architecture

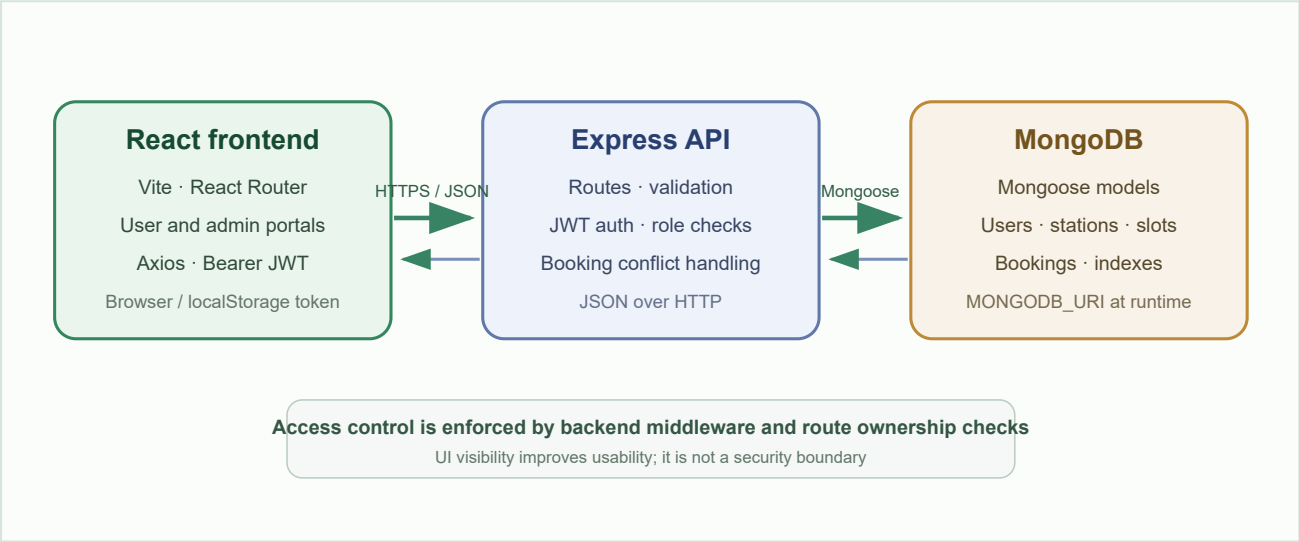


Figure 1. VoltSlot logical three-tier web architecture (derived from app setup and API client).

Components

- **Frontend:** React SPA using React Router for public, customer, and admin pages; Axios calls the API and adds a Bearer token from browser localStorage.
- **Backend:** Express app mounts authentication, customer/admin analytics, stations, nested slot, and booking routers under `/api`. Helmet, CORS policy, JSON body limit, JWT middleware, validation, and centralized error handling are configured in `backend/app.js`.
- **Database:** MongoDB collections are represented by Mongoose models. The server connects through `MONGODB_URI` before starting its listener.

Data flow

1. The browser submits a form or loads a page and calls the REST API with JSON; authenticated calls attach `Authorization: Bearer <token>`.
2. Express validates request values and, where configured, authenticates the JWT and checks a role.
3. Route handlers read or update Mongoose documents and perform booking/slot checks.
4. The API responds with JSON and a suitable status; React updates the page or renders an error.

5. System Modules

Authentication and profile

Registration creates customer accounts only; passwords must include uppercase, lowercase, and a digit and be 8–72 characters/bytes according to route validation. bcrypt hashes passwords at cost 12. Login verifies the hash and issues a JWT with role, subject, and one-hour expiry. The client restores a session through `GET /auth/me`; profile updates accept name and email.

Admin Portal

Protected admin routes include an operations dashboard, station list/form, slot tools on station details, and booking review. Admins can add/edit/delete stations, create/edit/activate/deactivate/delete slots subject to booking dependencies, filter bookings by status, cancel reservations, and mark ended confirmed bookings completed.

User Portal

Customer pages include booking metrics and activity, station browsing/details, My Bookings, and profile editing. The shared slot view offers reserve controls to customer-role accounts; unauthenticated visitors are directed to sign in. Admin slot views expose management actions instead of reservation controls.

Station management

Station records contain address/city, coordinates, connector types, kWh price, opening hours, and active/inactive/maintenance status. Admin-only create/update/delete routes validate model data. Deletion is rejected while slot or booking records depend on a station.

Slot management

Each slot belongs to a station and has start/end timestamps and available/booked/unavailable status. Only admins create or edit slots. The API rejects invalid ranges and duplicate station/time intervals, and prevents deleting or rescheduling slots referenced by booking history.

Booking and cancellation

The reservation handler validates an ObjectId and positive finite energy estimate, atomically changes an available future slot to booked, requires an active station, computes the estimated total from station price, and creates a confirmed booking. A partial unique index provides a second guard against duplicate confirmed bookings for one slot. Customer cancellation is restricted to the booking owner, before the slot starts; cancellation releases the slot. Admins can cancel any eligible booking and can complete a confirmed booking only after the slot ends.

Dashboards and analytics

The customer dashboard aggregates only the token subject's booking data, status totals, upcoming count/list, and recent activity. Admin analytics aggregate station/slot counts, booking status totals, recent bookings, and top stations. Charts are rendered in React with Recharts.

Authorization and security

Route guards improve UI routing, but API middleware is the security boundary. Administrator station/slot/dashboard endpoints require admin role; customer dashboard and booking creation require customer role. Booking reads and cancellation check ownership or administrator status.

6. Database Design

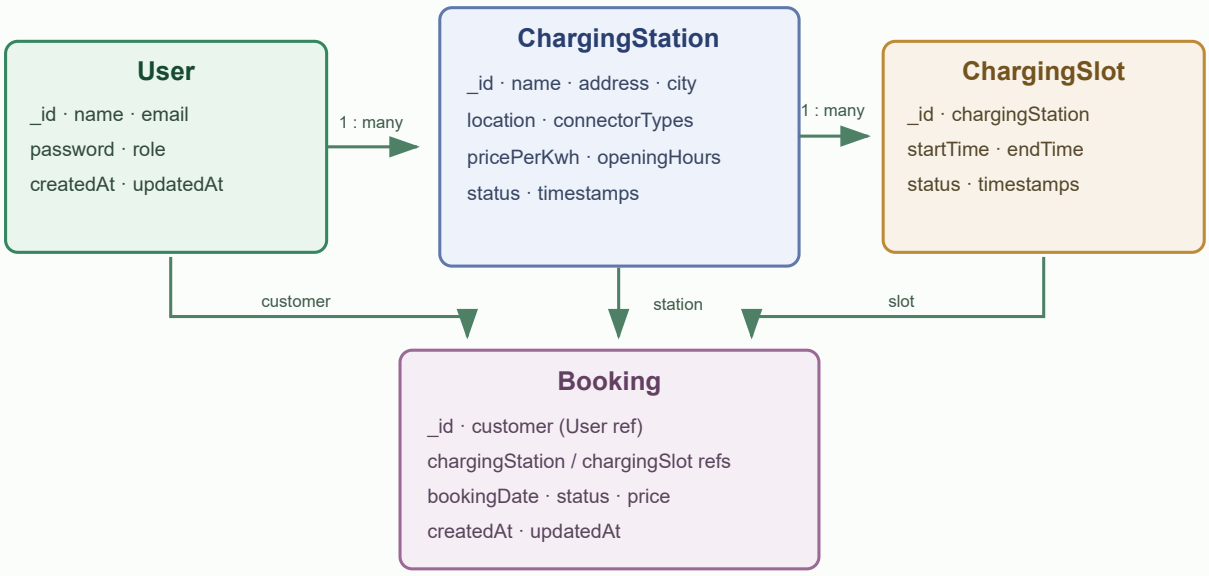


Figure 2. Entity relationships from Mongoose references. A confirmed slot can have at most one booking at a time.

Model / collection	Fields (type; constraints/defaults)	Relations / indexes
User `users`	`name` String required, trimmed, 2–100; `email` String required, unique, lowercase, valid email, max 254; `password` String required, select:false; `role` enum customer/admin, default customer; timestamps.	Created by registration or administrator seed. Password hashed with bcrypt cost 12 in registration and model save hook.
ChargingStation `chargingstations`	`name` String required, max 150; `description` String optional, max 2000; `address` String required, max 300; `city` String required, max 100; `location` GeoJSON Point with [longitude, latitude] bounds; `connectorTypes` nonempty enum list; `pricePerKwh` Number >= 0; `openingHours` required opens/closes HH:mm; `status` active/inactive/maintenance default active; timestamps.	2dsphere index on location; compound index city/status. Opening hours is embedded without its own _id.
ChargingSlot `chargingslots`	`chargingStation` ObjectId ref required; `startTime`, `endTime` Date required and end later than start; `status` available/booked/unavailable default available; timestamps.	Many slots belong to one station. Unique compound index station + start + end.
Booking `bookings`	`customer`, `chargingStation`, `chargingSlot` ObjectId refs required; `bookingDate` Date required default now; `status` confirmed/cancelled/completed default confirmed; `price` object with pricePerKwh, energyKwh, totalAmount (Numbers >=0), currency (uppercase three-letter code default USD); timestamps.	References one user, one station, one slot. Indexes customer/date and slot/status; partial unique index allows at most one confirmed booking per slot.

Relationships and integrity

- One station can have many charging slots.
- Each booking references one customer account, one station, and one charging slot. The schema stores ObjectId references; the API populates or joins display details when returning bookings.
- Customer booking lists filter by the authenticated user ID rather than accepting an arbitrary customer ID.

- A unique partial index on `chargingSlot` where status is `confirmed` prevents multiple confirmed bookings per slot. A slot/time compound unique index prevents duplicate schedules within the same station.
- Station deletion is blocked when slots or bookings reference it. Slot schedule changes/deletion are blocked when any historical booking depends on the slot.

7. API Documentation

All API paths below are prefixed with `/api`. The server returns JSON. “JWT” means `Authorization: Bearer <token>`. Roles use the backend values `customer` and `admin`.

Method	Endpoint	Purpose	Authentication / authorization	Success / notable result
GET	/api/health	API and database readiness	None	200 or 503
POST	/api/auth/register	Create customer account	None	201
POST	/api/auth/login	Authenticate and issue JWT	None	200
GET	/api/auth/me	Get current account	Any valid JWT	200
PATCH	/api/auth/me	Update own name/email	Any valid JWT	200
GET	/api/user/dashboard	Customer booking summary and activity	customer	200
GET	/api/admin/dashboard	Station, slot and booking analytics	admin	200
GET	/api/stations	List/filter stations and upcoming availability	None	200
GET	/api/stations/:id	Get one station	None	200
POST	/api/stations	Create station	admin	201
PUT	/api/stations/:id	Update station	admin	200
DELETE	/api/stations/:id	Delete station without dependent slots/bookings	admin	200 / 409
GET	/api/stations/:stationId/slots	List slots in start-time order	None	200
POST	/api/stations/:stationId/slots	Create slot	admin	201
PUT	/api/stations/:stationId/slots/:slotId	Update schedule/status	admin	200 / 409
DELETE	/api/stations/:stationId/slots/:slotId	Delete slot without booking history	admin	200 / 409
POST	/api/bookings	Claim available future slot and create booking	customer (JWT)	201 / 403 / 409
GET	/api/bookings/my	List bookings for token subject	Any valid JWT	200

Method	Endpoint	Purpose	Authentication / authorization	Success / notable result
GET	/api/bookings	List all bookings	admin	200
GET	/api/bookings/:bookingId	Read own booking or any booking as admin	Owner or admin	200 / 403
PATCH	/api/bookings/:bookingId/cancel	Cancel eligible owner booking; admin may cancel any	Owner or admin	200 / 403 / 409
PATCH	/api/bookings/:bookingId/complete	Mark ended confirmed booking completed	admin	200 / 409

Request payloads (field summary)

Operation	Body / query fields evidenced by handlers
Register	`{ name, email, password }`; optional role is accepted only as customer.
Login	`{ email, password }`.
Profile PATCH	`{ name?, email? }`; at least one allowed field and no other fields.
Station create/update	Editable fields: `name, description, address, city, location, connectorTypes, pricePerKwh, openingHours, status`. `location` uses GeoJSON Point coordinates [longitude, latitude].
Slot create/update	Create requires ISO `startTime` and `endTime`; update accepts schedule and/or `status: available|unavailable`. Booking transitions control booked status.
Booking create	`{ slotId, energyKwh }`; energyKwh must be a finite number greater than zero.
Station list query	`city, connectorType, status, availability, search, longitude, latitude, maxDistance`; maxDistance is in meters, 1–50,000. Coordinates must be supplied together.
Cancel / complete	No request body required; booking ID is a URL parameter.

Representative response shapes

```
POST /api/auth/login → 200
{ "message": "Login successful.", "token": "<JWT>", "user": { "id": "...", "name": "...", "email": "...",
"role": "customer" } }

GET /api/stations → 200
{ "stations": [{ "_id": "...", "name": "...", "availableSlots": 2 }] }

POST /api/bookings → 201
{ "message": "Charging slot reserved successfully.", "booking": { "status": "confirmed", "price": {
"totalAmount": 8.4 } }, "slot": { "durationMinutes": 60 } }
```

Common errors

`400` malformed/invalid input or IDs; `401` missing, invalid, expired, or no-longer-valid account token; `403` role/ownership denied; `404` unknown station, slot, booking, or account; `409` duplicate/conflicting schedule,

reservation conflict, dependent record, or ineligible transition; `413` JSON body exceeds 100 KB; `429` login/registration rate limit; `500` unexpected or configuration error; `503` health endpoint when database is disconnected.

> **Implementation-specific authorization note:** `POST /api/bookings` uses `authenticateToken` but does not call `authorizeRoles('customer')`. The frontend currently shows reservation controls only to signed-in customer-role accounts, and the booking is associated with the JWT subject; however, the frontend role gate is not a security control. The code does not establish that direct API calls from an authenticated admin are rejected for role alone. This report records that limitation rather than overstating server enforcement.

8. Implementation Details

Frontend structure

Area	Source locations / responsibilities
Routing and shells	`frontend/src/App.jsx` defines public, admin, customer and authenticated routes; `components/*PortalLayout.jsx` render role-specific navigation.
Pages	`pages/` contains dashboards, station directory/details, login/register, booking history, profile, admin station maintenance, and admin booking management.
Reusable UI	`components/` contains station and booking cards, station form, slot list/editor, loading/error components and layouts.
Authentication/API	`auth/AuthContext.jsx` manages account state; `api/client.js` configures Axios, bearer token attachment and 401 session clearing.
Styling and tests	`App.css`, `main.jsx`, Vitest/React Testing Library tests in `src/App.test.jsx`; Playwright specs under `frontend/e2e/`.

Backend structure

Area	Source locations / responsibilities
Server and app	`backend/server.js` loads environment configuration, connects MongoDB, then listens; `app.js` configures middleware, mounts API routes and handles errors.
Routes	`routes/auth.js`, `user.js`, `admin.js`, `stations.js`, `slots.js`, `bookings.js`.
Models	`models/User.js`, `ChargingStation.js`, `ChargingSlot.js`, `Booking.js`.
Auth/data	`middleware/auth.js` verifies JWTs and checks role; `config/db.js` requires MONGODB_URI; `scripts/seedAdmin.js` creates an initial admin from configured environment variables.
Tests	Node test runner, Supertest, and model/route tests under `backend/*.test.js`.

Authentication flow

1. Registration or login validates credentials and creates a one-hour signed JWT.
2. Frontend saves the token in localStorage and stores the returned public user.

3. Axios sends the token as a bearer header. On startup, the app requests `/auth/me` to restore session state.
4. Backend middleware verifies signature/expiry and validates subject and role. A 401 response clears client token and dispatches an unauthorized event.
5. Protected React routes redirect based on loading state, authentication, and role; API middleware remains authoritative.

Booking workflow

1. Customer opens a station and loads the public schedule.
2. Only a customer-role session receives the Reserve control in the frontend; energy estimate is collected.
3. Backend validates input, atomically claims an available future slot, checks station status, calculates amount, and saves a confirmed booking referencing customer/station/slot.
4. Client displays confirmation and refreshes availability. Conflict/validation responses are surfaced to the user.
5. Customer can cancel a confirmed booking before its start time; API checks ownership and releases the slot.

Admin management workflow

An authenticated admin opens the administration portal, creates or edits station information, follows the station's slot-management action, and schedules or activates slots. Admin booking management lists all bookings and exposes eligible cancellation/completion operations. The UI and backend avoid deleting station/slot records that have dependent booking records.

MongoDB integration

Mongoose connects using `MONGODB_URI`; the server does not start listening until the connection succeeds. Models define validation, timestamps, references, and indexes. Automated backend tests mock Mongoose methods and avoid modifying live Atlas records.

9. User Manual

Customer

1. Open the frontend and choose customer registration or sign in. Registration creates a customer account; use the credentials configured for your deployment.
2. Browse the public directory or choose *Find stations* in the User Portal. Search by text/city and optionally filter connector, station status, slot availability, or coordinates when supported.
3. Open a station to see details and scheduled slots. Sign in as a customer to reserve an available future slot.
4. Select Reserve, enter estimated energy in kWh, then confirm. Review the confirmation and estimated price.
5. Open *My bookings* to review status and schedule. Cancel an eligible confirmed booking before its start; confirm the cancellation prompt.
6. Open *My profile* to update name/email, or use Sign out to clear the browser session.

Administrator

1. Sign in through the Admin Portal using an administrator account. Public registration cannot create an administrator account; the repository includes a guarded seed script requiring administrator credentials in local environment configuration.
2. Use Dashboard for station/slot/booking counts, booking-status analytics, top booked stations, and recent activity.
3. Open Stations & slots. Add a station or edit an existing station's location, connectors, pricing, hours, and operational status.
4. Choose Manage slots for a station. Create future slots, edit allowed schedules, activate/deactivate or delete eligible slots. Existing bookings can block certain changes/deletions.
5. Open Manage bookings to filter bookings by status, cancel eligible bookings, or mark an ended confirmed booking completed.
6. Sign out when finished.

The exact visible navigation label on station cards/details is “View details”; admin station rows provide “Manage slots”; the portal provides “Stations & slots” and “Manage bookings”.

Screenshot insertion points

Screenshots are intentionally placeholders: no screenshot was taken using synthetic data or by mutating the configured database. Replace these boxes with genuine application captures if an isolated demo environment is available.

[SCREENSHOT PLACEHOLDER: Landing page and station listing]
Insert genuine capture of the running public directory / landing page.

[SCREENSHOT PLACEHOLDER: Customer dashboard and slot reservation]
Insert genuine customer account capture: dashboard, station schedule, and confirmation. Do not show secrets or another customer’s data.

[SCREENSHOT PLACEHOLDER: My Bookings]
Insert genuine customer booking-history capture from a safe demo account.

[SCREENSHOT PLACEHOLDER: Admin dashboard and management]
Insert genuine administrator dashboard plus station/slot management captures from a safe demo environment.

10. Testing and Results

Results below were executed for this documentation task, in the current local checkout. Browser E2E tests use mocked/stubbed API responses; the backend tests use isolated mocked database methods, as documented in the repository runtime validation report. They do not represent live Atlas mutations or end-to-end persistence.

Verification	Command	Observed result
Backend automated suite	`cd backend; npm test`	**PASS** · 71 tests, 18 suites; 71 passed, 0 failed, 0 skipped. Node built-in test runner.
Frontend unit/component suite	`cd frontend; npm exec vitest run` (part of npm test)	**PASS** · 31 tests, 1 test file; all 31 passed.
Frontend browser E2E	`cd frontend; npm test` (Playwright phase)	**PASS** · 8 tests passed; customer registration/roles/login/profile/logout, dashboards, station/slot admin operations, customer booking/reservation, admin booking review/cancellation, and mobile navigation. API requests are stubbed.
Frontend production build	`cd frontend; npm run build`	**PASS** · Vite 8.3.4 production build; 680 modules transformed; build completed successfully.
Live database CRUD / browser screenshots	Not run	Not claimed. No database mutation was performed for this documentation task; screenshot placeholders are provided.

Verification iteration: the first Playwright run had 7 passing tests and 1 failure because an invalid-login response was displayed as an expired-session message. The login page was corrected to show the API's invalid-credentials message; the final full frontend test command then passed all 31 unit tests and all 8 browser tests.

Representative verified test cases

Case	Input / setup	Expected behavior	Observed
Registration	Valid customer fields; disallowed admin role test	Hash password; return public user/JWT; reject public admin assignment.	Pass in backend auth tests.
Authorization	Missing/expired token; customer to admin endpoint; admin to booking-create endpoint	401 for invalid session; 403 for forbidden role, without slot/booking mutation.	Pass in backend auth/admin/station/slot /booking tests.
Slot conflict	Attempt competing/repeated reservation of same slot	Only one claim succeeds; conflict response protects booked slot.	Pass in backend booking tests.

Case	Input / setup	Expected behavior	Observed
Booking ownership	Customer accesses or cancels own and another customer's booking	Own record permitted; another customer’s access denied.	Pass in backend booking tests.
Cancellation	Owner cancels future confirmed booking	Booking becomes cancelled; slot is released; started/nonconfirmed cases rejected.	Pass in backend booking tests.
Customer UI reservation	Signed-in customer with future available slot, energy 24 kWh	Reserve/confirm posts slot ID + energy and displays confirmation.	Pass in frontend unit and stubbed E2E tests.
Admin management UI	Admin creates/edits/deletes station/slot using stubbed API	Admin actions render and update; reserved slots remain protected.	Pass in Playwright admin-management tests.

No manual browser tests are claimed. The Playwright E2E suite is automated; its API calls are intercepted by test fixtures, so it verifies frontend behavior rather than live database behavior.

11. Security Considerations

- **Password storage:** bcrypt hashing with work factor 12; the User model excludes password from default queries and removes it from JSON.
- **JWT sessions:** JWT secret is required; tokens include subject and role and expire after one hour. Keep JWT_SECRET outside source control.
- **Role-based authorization:** middleware rejects missing/invalid bearer tokens and unauthorized roles. Reservation creation requires the customer role; admin-only station/slot operations and analytics are checked server-side.
- **Input constraints:** route validators, explicit editable-field allowlists, Mongoose constraints, ObjectId checks, request JSON limit of 100 KB, and CORS allowlist.
- **Ownership:** customer booking listing scopes to JWT user ID; detail and cancel actions enforce owner unless the account is admin.
- **Reservation conflicts:** atomic slot claim and a partial unique confirmed-booking index prevent competing confirmations. Changes to records with dependent history are restricted.
- **Request throttling / headers:** login and registration rate limits; Helmet security headers.
- **Secret handling:** report omits .env contents. Configure Mongo URI, JWT secret, and admin seed credentials locally/through a managed secret mechanism. Never commit credentials.

> **Booking authorization verified:** booking creation requires a customer JWT; administrator tokens receive 403 before a slot is claimed. UI visibility is not relied on as the security boundary.

12. Limitations and Future Enhancements

Observed limitations

- No payment processing, refund, or invoice module appears in the inspected routes/models/UI.

- No push/email/SMS notifications or live station telemetry are implemented in the inspected application sources.
- Station and booking list routes do not expose pagination in the inspected handlers, so large datasets may need future query limits.
- Automated backend route tests use mocked database operations; real deployment-level concurrency and index behavior require a separate isolated MongoDB integration test.
- Browser E2E tests use stubbed API responses and do not prove persistence against a live backend/database.
- The frontend does not include live captured screenshots in this submission; labelled insertion areas are provided.
- The data model uniquely prevents duplicate confirmed bookings for the same slot ID, but it does not reject reservations for distinct slot records whose time intervals overlap. Station-level capacity/charger semantics are not modelled.

Potential future work (not implemented)

- If overlapping intervals should conflict, define station charger/capacity semantics and implement database-safe exclusion for distinct slot records.
- Add isolated MongoDB integration tests and concurrency testing against a disposable test database.
- Add pagination/sorting controls for larger directories and booking histories.
- Add optional payment integration, booking notifications, maps/route planning, and charger status telemetry after separate requirements and security review.
- Introduce deployment automation, observability, and operational backups appropriate to the eventual hosting environment.

13. Conclusion

VoltSlot is a MERN application that connects station discovery, scheduled slot administration, customer bookings, and operational oversight. Its implementation demonstrates reusable React pages, Express REST routes, Mongoose document relationships, role-aware UI, API authentication and authorization, input validation, and automated checks for booking conflicts and ownership. The executed test suites and production build provide evidence for local code-level behavior; mocked test boundaries and the lack of station-level exclusion for distinct overlapping slots should remain visible when planning deployment or further security work.

14. References

1. React documentation, https://react.dev/learn
2. Node.js documentation, https://nodejs.org/docs/latest/api/
3. Express documentation, https://expressjs.com/
4. MongoDB manual, https://www.mongodb.com/docs/manual/
5. Mongoose documentation, https://mongoosejs.com/docs/
6. JSON Web Token introduction, https://jwt.io/introduction. VoltSlot uses the `jsonwebtoken` npm package to sign and verify bearer tokens.
7. Vite documentation, https://vite.dev/guide/

8. Bootstrap documentation, https://getbootstrap.com/docs/5.3/

References are vendor/project documentation; retrieved for this report on 2026-10-14. The implementation-specific descriptions in this report are based on the source code in this repository.

Documentation generator source: `scripts/generate-documentation.cjs`. Report generated 2026-10-14.